

8086 Assembly Language Programs

Week #2 — Lab #2

Byte and Word Data Transfer in Different Addressing Modes

```
DATA SEGMENT
    DATA1 DB 23H          ; byte variable
    DATA2 DW 1234H        ; word variable
    DATA3 DB 0H           ; byte variable, result storage
    DATA4 DW 0H           ; word variable, result storage
    DATA5 DW 2345H, 6789H ; array of two words
DATA ENDS

CODE SEGMENT
    ASSUME CS:CODE, DS:DATA

START: MOV AX, DATA      ; initialize DS to point to start of data segment
       MOV DS, AX
       MOV AL, 25H        ; immediate addressing: copy 25H into 8-bit AL
       MOV AX, 2345H      ; immediate addressing: copy 2345H into 16-bit AX
       MOV BX, AX         ; register addressing: copy AX into BX
       MOV CL, AL         ; register addressing: copy AL into CL

       MOV AL, DATA1     ; direct addressing: byte from DATA1 into AL
       MOV AX, DATA2     ; direct addressing: word from DATA2 into AX
       MOV DATA3, AL     ; direct addressing: AL into byte DATA3
       MOV DATA4, AX     ; direct addressing: AX into word DATA4

       MOV BX, OFFSET DATA5 ; BX = offset address of DATA5
       MOV AX, [BX]       ; register-indirect addressing: word at [BX] into AX

       MOV DI, 02H        ; index value
       MOV AX, [BX+DI]    ; base + index addressing: word at [BX+DI] into AX
       MOV AX, [BX+0002H] ; base + displacement addressing
       MOV AL, [DI+2]     ; register-relative addressing
       MOV AX, [BX+DI+0002H] ; base + index + displacement addressing

       MOV AH, 4CH        ; DOS function: terminate program
       INT 21H
CODE ENDS
END START
```

Block Interchange (Swap Two Arrays)

```
DATA SEGMENT
    X DB 01H,02H,03H,04H,05H ; source array
    Y DB 11H,12H,13H,14H,15H ; destination array
DATA ENDS

CODE SEGMENT
    ASSUME CS:CODE, DS:DATA

START: MOV AX, DATA
       MOV DS, AX
       MOV CX, 05H          ; loop counter = array length
```

```

        LEA SI, X           ; SI -> array X
        LEA DI, Y           ; DI -> array Y

UP:     MOV BL,[SI]         ; BL = X[i]
        MOV AL,[DI]         ; AL = Y[i]
        MOV [SI], AL        ; X[i] = old Y[i]
        MOV [DI], BL        ; Y[i] = old X[i]
        INC SI              ; advance to next element of X
        INC DI              ; advance to next element of Y
        DEC CX              ; decrement counter
        JNZ UP              ; repeat until all elements swapped

        MOV AH, 4CH         ; exit to DOS
        INT 21H

CODE ENDS
END START

```

Week #4 — Lab #4

Logical Operation — Program 1 (AND)

```

CODE SEGMENT
    ASSUME CS:CODE, DS:CODE

    MOV BX, 3256H           ; BX = 3256H
    MOV CX, 1554H           ; CX = 1554H
    AND CX, BX              ; CX = CX AND BX -> bitwise AND, observe result in CX
    HLT                     ; halt (use single-step/debugger to inspect CX)

CODE ENDS
END

```

Logical Operation — Program 2 (XOR)

```
CODE SEGMENT
    ASSUME CS:CODE, DS:CODE

    MOV BX, 3256H      ; BX = 3256H
    MOV CX, 1554H      ; CX = 1554H
    XOR CX, BX          ; CX = CX XOR BX -> bits differ = 1, bits same = 0
    HLT                ; halt; observe CX in debugger

CODE ENDS
END
```

Logical Operation — Program 3 (OR, XOR, NOT, TEST, AND)

```
CODE SEGMENT
    ASSUME CS:CODE, DS:CODE

    MOV AX, 1027H      ; AX = 1027H
    MOV BX, 5A27H      ; BX = 5A27H
    MOV CX, 54A5H      ; CX = 54A5H
    OR  AX, BX          ; AX = AX OR BX
    XOR AX, CX          ; AX = AX XOR CX
    NOT AX              ; AX = one's complement of AX
    TEST CX, BX         ; CX AND BX, sets flags only, CX/BX unchanged
    AND CX, AX          ; CX = CX AND AX
    HLT                ; halt; step through and compare with hand calculation

CODE ENDS
END
```

Week #4 — Lab #4 (Part 2)

Program 1 — Branching with SUB/DIV/AND/TEST

```
CODE SEGMENT
    ASSUME CS:CODE, DS:CODE

    MOV AX, 7A24H
    MOV BX, 15A3H
    SUB AX, BX          ; AX = 7A24H - 15A3H
    JMP L3T2            ; jump forward, skipping EEE316 the first time

EEE316: DIV BX          ; unsigned divide: DX:AX / BX -> AX=quotient, DX=remainder
    JMP LAST

L3T2:  MOV CX, 45B1H
    AND AX, CX          ; AX = AX AND CX
    TEST AX, BX         ; flags only
    JMP EEE316          ; now execute the DIV above

LAST:  HLT              ; halt; single-step and record every register

CODE ENDS
END
```

Program 2 — Branching with ADD/OR/SUB and Conditional Jumps

```
CODE SEGMENT
    ASSUME CS:CODE, DS:CODE

    MOV AX, 7A24H
    MOV BX, 95A3H
    ADD AX, BX          ; AX = 7A24H + 95A3H
    JC L3T2             ; if carry, go to L3T2

EEE316: OR AX, 23H
        JNZ LAST        ; if result nonzero, exit

L3T2:   MOV CX, 0FC7H
        SUB AX, CX
        JZ EEE316        ; if zero, go back and OR/test again

LAST:   HLT              ; halt; single-step and record every register

CODE ENDS
END
```

16-bit Addition

```
DATA SEGMENT
    NUM DW 1234H, 0F234H      ; two 16-bit numbers to add
    SUM DW 2 DUP(0)          ; SUM (low word), SUM+2 (carry word)
DATA ENDS

CODE SEGMENT
    ASSUME CS:CODE, DS:DATA

START:  MOV AX, DATA
        MOV DS, AX
        MOV AX, NUM          ; AX = first number
        MOV BX, 0H           ; BX will hold the carry
        ADD AX, NUM+2         ; AX = AX + second number
        JNC DOWN             ; if no carry, skip
        INC BX               ; else record the carry
DOWN:   MOV SUM, AX           ; store 16-bit sum
        MOV SUM+2, BX        ; store carry-out
        MOV AH, 4CH
        INT 21H
CODE ENDS
END START

; INPUT : 1234H, F234H
; OUTPUT: 10468H
```

32-bit Addition (two 16-bit words per number)

```
DATA SEGMENT
    NUM1 DW 0FFFFH, 0FFFFH   ; NUM1 = FFFFFFFFH (low word, high word)
    NUM2 DW 1111H, 1111H     ; NUM2 = 11111111H
    SUM  DW 4 DUP(0)         ; result: low word, high word, carry word
DATA ENDS

CODE SEGMENT
    ASSUME CS:CODE, DS:DATA

START:  MOV AX, DATA
        MOV DS, AX
        MOV AX, NUM1         ; AX = low word of NUM1
        ADD AX, NUM2         ; add low word of NUM2
        MOV SUM, AX          ; store low word of result
        MOV AX, NUM1+2       ; AX = high word of NUM1
        ADC AX, NUM2+2        ; add high word of NUM2, with carry from low word
        JNC EXIT             ; check for carry-out of the 32-bit add
        MOV WORD PTR SUM+4, 01H ; store the carry-out
EXIT:   MOV SUM+2, AX         ; store high word of result
        MOV AH, 4CH
        INT 21H
CODE ENDS
END START

; INPUT : FFFFFFFFH, 11111111H
; OUTPUT: 11111110H (33-bit result, carry word = 1)
```

32-bit Addition Using Pointers (SI/DI)

```
DATA SEGMENT
    NUM1 DW 5678H, 1234H      ; NUM1 = 12345678H (low word, high word)
    NUM2 DW 5678H, 1234H      ; NUM2 = 12345678H
    SUM  DW 3 DUP(0)
DATA ENDS

CODE SEGMENT
    ASSUME CS:CODE, DS:DATA

START:  MOV AX, DATA
        MOV DS, AX
        LEA SI, NUM1          ; SI -> low word of NUM1
        LEA DI, NUM2          ; DI -> low word of NUM2
        MOV AX, [SI]           ; AX = low word of NUM1
        ADD AX, [DI]           ; AX += low word of NUM2
        MOV CX, [SI+2]         ; CX = high word of NUM1
        ADC CX, [DI+2]         ; CX += high word of NUM2 + carry
        JNC DOWN
        MOV WORD PTR SUM+4, 01H ; store carry-out
DOWN:   MOV SUM, AX             ; store low word
        MOV SUM+2, CX          ; store high word
        MOV AH, 4CH
        INT 21H
CODE ENDS
END START

; INPUT : 12345678H, 12345678H
; OUTPUT: 2468ACF0H
```

16-bit Subtraction

```
DATA SEGMENT
    NUM DW 4567H, 2345H      ; minuend, subtrahend
    DIF DW 1 DUP(0)
DATA ENDS

CODE SEGMENT
    ASSUME CS:CODE, DS:DATA

START:  MOV AX, DATA
        MOV DS, AX
        CLC                      ; clear carry/borrow flag
        LEA SI, NUM              ; SI -> NUM
        MOV AX, [SI]             ; AX = NUM[0]
        SBB AX, [SI+2]           ; AX = AX - NUM[1] - borrow
        MOV DIF, AX              ; store result
        MOV AH, 4CH
        INT 21H
CODE ENDS
END START

; INPUT : 4567H, 2345H
; OUTPUT: 2222H
```

32-bit Subtraction

```
DATA SEGMENT
    NUM1 DW 2345H, 6762H      ; minuend (low, high)
    NUM2 DW 1111H, 1111H      ; subtrahend (low, high)
    DIF  DW 2 DUP(0)
DATA ENDS

CODE SEGMENT
    ASSUME CS:CODE, DS:DATA

START:  MOV AX, DATA
        MOV DS, AX
        LEA SI, NUM1           ; SI -> low word of NUM1
        LEA DI, NUM2           ; DI -> low word of NUM2
        MOV AX, [SI]
        MOV BX, [DI]
        SUB AX, BX              ; AX = low(NUM1) - low(NUM2)
        MOV DIF, AX            ; store low result
        INC SI                  ; advance SI to high word of NUM1
        INC SI
        INC DI                  ; advance DI to high word of NUM2
        INC DI
        MOV AX, [SI]
        MOV BX, [DI]
        SBB AX, BX              ; AX = high(NUM1) - high(NUM2) - borrow
        MOV DIF+2, AX          ; store high result
        MOV AH, 4CH
        INT 21H
CODE ENDS
END START
```

Week #6 — Lab #6

16-bit Multiplication — Unsigned (MUL)

```
DATA SEGMENT
    NUM DW 1234H, 1234H      ; multiplicand, multiplier
    PROD DW 2 DUP(0)
DATA ENDS

CODE SEGMENT
    ASSUME CS:CODE, DS:DATA

START:  MOV AX, DATA
        MOV DS, AX
        LEA SI, NUM
        MOV AX,[SI]          ; multiplicand into AX (required by MUL)
        MOV BX,[SI+2]        ; multiplier into BX
        MUL BX               ; unsigned multiply: DX:AX = AX * BX
        MOV PROD, AX         ; low word of product
        MOV PROD+2, DX       ; high word of product
        MOV AH, 4CH
        INT 21H
CODE ENDS
END START

; INPUT : 1234H x 1234H
; OUTPUT: DX=014BH, AX=5490H
```

16-bit Multiplication — Signed (IMUL)

```
DATA SEGMENT
    NUM DW -2, 1             ; multiplicand = -2, multiplier = 1
    PROD DW 2 DUP(0)
DATA ENDS

CODE SEGMENT
    ASSUME CS:CODE, DS:DATA

START:  MOV AX, DATA
        MOV DS, AX
        LEA SI, NUM
        MOV AX,[SI]          ; multiplicand
        MOV BX,[SI+2]        ; multiplier
        IMUL BX              ; signed multiply: DX:AX = AX * BX
        MOV PROD, AX
        MOV PROD+2, DX
        MOV AH, 4CH
        INT 21H
CODE ENDS
END START

; INPUT : -2 x 1
; OUTPUT: DX=FFFFH, AX=FFFEH (result in two's complement)
```


8-bit Division — Unsigned (DIV)

```
DATA SEGMENT
    NUM1 DB 72H          ; dividend
    NUM2 DB 02H          ; divisor
    QUO  DB 1 DUP(0)
    REM  DB 1 DUP(0)
DATA ENDS

CODE SEGMENT
    ASSUME CS:CODE, DS:DATA

START:  MOV AX, DATA
        MOV DS, AX
        MOV AL, NUM1      ; dividend into AL
        MOV AH, 0H        ; zero-extend to 16 bits for 16/8 division
        DIV NUM2          ; unsigned divide: AX / NUM2
        MOV QUO, AL       ; quotient
        MOV REM, AH       ; remainder
        MOV AH, 4CH
        INT 21H
CODE ENDS
END START

; INPUT : 72H / 02H -> AL=39H, AH=00H
; INPUT : 55H / 04H -> AL=15H, AH=01H
```

8-bit Division — Signed (IDIV)

```
DATA SEGMENT
    NUM1 DB -10          ; dividend
    NUM2 DB 02           ; divisor
    QUO  DB 1 DUP(0)
    REM  DB 1 DUP(0)
DATA ENDS

CODE SEGMENT
    ASSUME CS:CODE, DS:DATA

START:  MOV AX, DATA
        MOV DS, AX
        MOV AL, NUM1      ; dividend into AL
        CBW              ; sign-extend AL into AX
        IDIV NUM2         ; signed divide
        MOV QUO, AL
        MOV REM, AH
        MOV AH, 4CH
        INT 21H
CODE ENDS
END START

; INPUT : -10 / 02 -> AL=FBH (result in two's complement)
; INPUT : -10 / 03 -> AL=FDH, AH=FFH
```

16-bit Division — Unsigned (DIV)

```
DATA SEGMENT
    NUM1 DW 4567H, 2345H ; 32-bit dividend (low word, high word)
    NUM2 DW 4111H        ; divisor
    QUO  DW 2 DUP(0)
    REM  DW 1 DUP(0)
DATA ENDS
```

```

CODE SEGMENT
    ASSUME CS:CODE, DS:DATA

START:  MOV AX, DATA
        MOV DS, AX
        MOV AX, NUM1           ; low word of dividend
        MOV DX, NUM1+2         ; high word of dividend
        DIV NUM2               ; unsigned divide: DX:AX / NUM2
        MOV QUO, AX            ; quotient
        MOV REM, DX            ; remainder
        MOV AH, 4CH
        INT 21H

CODE ENDS
END START

; INPUT : 23454567H / 4111H -> AX=8AC5H, DX=0952H

```

16-bit Division — Signed (IDIV)

```

DATA SEGMENT
    NUM1 DW 4567H, 2345H
    NUM2 DW 4111H
    QUO  DW 2 DUP(0)
    REM  DW 1 DUP(0)
DATA ENDS

CODE SEGMENT
    ASSUME CS:CODE, DS:DATA

START:  MOV AX, DATA
        MOV DS, AX
        MOV AX, NUM1           ; low word of dividend
        MOV DX, NUM1+2         ; high word of dividend
        CWD                    ; (kept for reference; here DX is already set explicitly above)
        IDIV NUM2              ; signed divide
        MOV QUO, AX
        MOV REM, DX
        MOV AH, 4CH
        INT 21H

CODE ENDS
END START

```

Week #7 — Lab #7

Square and Cube of a Number

```

DATA SEGMENT
    X      DW 04H
    SQUARE DW ?
    CUBE   DW ?
DATA ENDS

CODE SEGMENT
    ASSUME CS:CODE, DS:DATA

START:  MOV AX, DATA
        MOV DS, AX
        MOV AX, X              ; AX = X
        MOV BX, X              ; BX = X
        MUL BX                 ; AX = X * X
        MOV SQUARE, AX         ; store square

```

```

        MUL BX                ; AX = (X*X) * X = X^3
        MOV CUBE, AX          ; store cube
        MOV AH, 4CH
        INT 21H
CODE ENDS
END START

; INPUT : X = 4H
; OUTPUT: SQUARE = 10H, CUBE = 40H

```

Factorial of a Number

```

DATA SEGMENT
    X    DW 06H
    FACT DW ?
DATA ENDS

CODE SEGMENT
    ASSUME CS:CODE, DS:DATA

START:  MOV AX, DATA
        MOV DS, AX
        MOV AX, 01H          ; running product, starts at 1
        MOV CX, X             ; CX = X (loop/multiplier counter)
UP:     MUL CX                ; AX = AX * CX
        LOOP UP               ; decrement CX, repeat while CX != 0
        MOV FACT, AX          ; store factorial
        MOV AH, 4CH
        INT 21H
CODE ENDS
END START

; INPUT : 06H
; OUTPUT: 2D0H (720 decimal)

```

Week #8 — Lab #8

LCM of Two Numbers

```

DATA SEGMENT
    NUM DW 05, 04
    LCM DW 2 DUP(0)
DATA ENDS

CODE SEGMENT
    ASSUME CS:CODE, DS:DATA

START:  MOV AX, DATA
        MOV DS, AX
        MOV DX, 0H           ; clear high word / running carry
        MOV AX, NUM           ; AX = first number
        MOV BX, NUM+2         ; BX = second number
UP:     PUSH AX                ; save running value (quotient side)
        PUSH DX               ; save running carry
        DIV BX                 ; AX = AX/BX, DX = remainder
        CMP DX, 0             ; is remainder zero?
        JE EXIT               ; if so, we have found the LCM
        POP DX                ; restore running carry
        POP AX                ; restore running value
        ADD AX, NUM            ; add first number again (repeated addition)
        JNC DOWN              ; check for carry out of 16 bits
        INC DX                 ; propagate the carry
DOWN:   JMP UP

```

```

EXIT:  POP LCM+2          ; store high word of LCM
        POP LCM           ; store low word of LCM
        MOV AH, 4CH
        INT 21H
CODE ENDS
END START

; INPUT : 0AH, 04H
; OUTPUT: LCM = ... (see debugger; original manual lists 02H, which reflects
;           intermediate remainder, not the final LCM value)

```

GCD of Two Numbers

```

DATA SEGMENT
    NUM1 DW 000AH
    NUM2 DW 0004H
    GCD  DW ?
DATA ENDS

CODE SEGMENT
    ASSUME CS:CODE, DS:DATA

START:  MOV AX, DATA
        MOV DS, AX
        MOV AX, NUM1
        MOV BX, NUM2
UP:     CMP AX, BX          ; compare the two numbers
        JE EXIT            ; equal -> BX (=AX) is the GCD
        JB EXCG            ; AX < BX -> swap so AX is always the larger
UP1:    MOV DX, 0H         ; clear DX before division
        DIV BX             ; AX = AX/BX, DX = remainder
        CMP DX, 0          ; remainder 0 -> BX is the GCD
        JE EXIT            ; else continue with the remainder
        MOV AX, DX
        JMP UP
EXCG:   XCHG AX, BX        ; swap so the larger value is in AX
        JMP UP1
EXIT:   MOV GCD, BX        ; store result
        MOV AH, 4CH
        INT 21H
CODE ENDS
END START

; INPUT : 0AH, 04H
; OUTPUT: GCD = 02H

```

Week #9 — Lab #9

Test Whether a Number Is Positive or Negative

```

DATA SEGMENT
    NUM  DB 12H
    MES1 DB 10,13,'DATA IS POSITIVE $'
    MES2 DB 10,13,'DATA IS NEGATIVE $'
DATA ENDS

CODE SEGMENT
    ASSUME CS:CODE, DS:DATA

START:  MOV AX, DATA
        MOV DS, AX
        MOV AL, NUM        ; number to test
        ROL AL, 1          ; rotate MSB (sign bit) into carry flag

```

```

        JC NEGA                ; CF=1 -> number is negative
        MOV DX, OFFSET MES1    ; else point to "positive" message
        JMP EXIT
NEGA:    MOV DX, OFFSET MES2    ; point to "negative" message
EXIT:    MOV AH, 09H           ; DOS: display string
        INT 21H
        MOV AH, 4CH
        INT 21H
CODE ENDS
END START

; Positive numbers: 00H-7FH   Negative numbers: 80H-FFH
; OUTPUT for NUM=12H: DATA IS POSITIVE

```

Test Whether a Number Is Odd or Even

```

DATA SEGMENT
    X      DW 27H
    MSG1 DB 10,13,'NUMBER IS EVEN$'
    MSG2 DB 10,13,'NUMBER IS ODD$'
DATA ENDS

CODE SEGMENT
    ASSUME CS:CODE, DS:DATA

START:  MOV AX, DATA
        MOV DS, AX
        MOV AX, X
        TEST AX, 01H          ; test bit 0 (LSB): 1 = odd, 0 = even
        JNZ EXIT              ; bit0=1 (odd) -> go print "ODD"
        LEA DX, MSG1          ; bit0=0 -> number is even
        MOV AH, 09H
        INT 21H
        JMP LAST
EXIT:    LEA DX, MSG2          ; number is odd
        MOV AH, 09H
        INT 21H
LAST:    MOV AH, 4CH
        INT 21H
CODE ENDS
END START

; INPUT : 27H
; OUTPUT: NUMBER IS ODD

```

Week #10 — Lab #10

ASCII Adjustment Instructions (AAA, AAD, AAM, AAS)

```

CODE SEGMENT
    ASSUME CS:CODE

START:  MOV AX, 31H           ; ASCII '1'
        ADD AX, 39H          ; + ASCII '9'
        AAA                  ; ASCII-adjust after addition: AX=0100H (unpacked BCD)
        ADD AX, 3030H        ; convert unpacked BCD digits to ASCII

        MOV BL, 9            ; divisor
        MOV AX, 0702H        ; two-digit unpacked BCD dividend (AAD needs this form)
        AAD                  ; adjust BEFORE division: converts unpacked BCD in AX to binary
        DIV BL               ; AX(binary) / BL -> AL=quotient, AH=remainder

        MOV AL, 5            ; multiplicand

```

```

        MOV CL, 5          ; multiplier
        MUL CL             ; AX = 0019H (25 decimal, binary)
        AAM                ; adjust AFTER multiply: AX=0205H (unpacked BCD)
        ADD AX, 3030H      ; AX=3235H, ASCII "25"

        MOV AX, 38H        ; ASCII '8'
        SUB AX, 31H        ; - ASCII '1'
        AAS                ; ASCII-adjust after subtraction: AX=0007H
        ADD AX, 3030H      ; AX=3037H, ASCII '7'

        MOV AH, 4CH
        INT 21H

CODE ENDS
END START

```

Binary to BCD Conversion

```

DATA SEGMENT
    BIN DW 01A9H
    BCD DB 2 DUP(0)
DATA ENDS

CODE SEGMENT
    ASSUME CS:CODE, DS:DATA

START:  MOV AX, DATA
        MOV DS, AX
        MOV AX, BIN          ; binary value to convert
        MOV CL, 64H          ; 100 decimal
        DIV CL                ; AL = value/100, AH = value%100
        MOV BCD+1, AL         ; hundreds digit
        MOV AL, AH            ; remainder (0-99) into AL
        MOV AH, 00H
        MOV CL, 0AH          ; 10 decimal
        DIV CL                ; AL = tens digit, AH = units digit
        MOV CL, 04           ; rotate count
        ROR AL, CL            ; move tens digit into upper nibble
        ADD AL, AH            ; combine tens (high nibble) + units (low nibble) -> packed BCD
        MOV AH, 4CH
        INT 21H

CODE ENDS
END START

; INPUT : 01A9H (425 decimal)
; OUTPUT: BCD = 425H

```

BCD to Binary Conversion

```

DATA SEGMENT
    BCD DW 27H
    BIN DW ?
DATA ENDS

CODE SEGMENT
    ASSUME CS:CODE, DS:DATA

START:  MOV AX, DATA
        MOV DS, AX
        MOV AX, BCD
        AND AX, 07H          ; isolate the units digit (assumes units <= 7 here)
        MOV BX, AX           ; save units digit
        MOV AX, BCD
        AND AX, 0F0H         ; isolate the tens digit (still in upper nibble position)
        MOV CX, 0AH          ; 10 decimal

```

```

        MUL CX                ; multiply upper-nibble value by 10
        ADD AX, BX            ; add units digit -> binary value
        MOV BIN, AX
        MOV AH, 4CH
        INT 21H
CODE ENDS
END START

; INPUT : BCD = 27H
; OUTPUT: BIN = 1BH (27 decimal)

```

Week #11 — Lab #11

Addition of N Numbers

```

DATA SEGMENT
    ARR DW 0010H,0020H,0030H,0040H,0050H
    LEN EQU ($-ARR)/2        ; number of words in ARR
    SUM DW ?
DATA ENDS

CODE SEGMENT
    ASSUME CS:CODE, DS:DATA

START:  MOV AX, DATA
        MOV DS, AX
        LEA SI, ARR          ; SI -> first element
        CLC                  ; clear carry flag
        XOR AX, AX           ; AX = 0 (also clears CF)
        MOV CX, LEN          ; loop counter = number of elements
UP:     ADC AX,[SI]            ; add element (with carry) to running sum
        INC SI                ; advance to next word (2 bytes)
        INC SI
        DEC CX
        JNZ UP
        MOV SUM, AX          ; store result
        MOV AH, 4CH
        INT 21H
CODE ENDS
END START

; OUTPUT: 00F0H

```

Subtraction of N Numbers

```

DATA SEGMENT
    ARR DW 50H,10H,20H,10H,05H
    LEN EQU ($-ARR)/2
    DIF DW ?
DATA ENDS

CODE SEGMENT
    ASSUME CS:CODE, DS:DATA

START:  MOV AX, DATA
        MOV DS, AX
        LEA SI, ARR
        CLC
        MOV CX, LEN-1        ; N-1 subtractions for N numbers
        MOV AX,[SI]          ; AX = first number
UP:     SUB AX,[SI+2]          ; AX -= next number
        INC SI
        INC SI

```

```

        DEC CX
        JNZ UP
        MOV DIF, AX
        MOV AH, 4CH
        INT 21H
CODE ENDS
END START

; OUTPUT: 0005H

```

Week #12 — Lab #12

Largest Number in a Series

```

DATA SEGMENT
    X    DW 0010H,52H,30H,40H,50H
    LAR DW ?
DATA ENDS

CODE SEGMENT
    ASSUME CS:CODE, DS:DATA

START:  MOV AX, DATA
        MOV DS, AX
        MOV CX, 05H           ; number of elements
        LEA SI, X
        MOV AX,[SI]           ; AX = running maximum, starts at X[0]
        DEC CX                ; N-1 comparisons remain
UP:     CMP AX,[SI+2]          ; compare running max with next element
        JA CTINUE             ; if AX already larger, keep it
        MOV AX,[SI+2]         ; else update running max
CTINUE: ADD SI, 2              ; advance to next element
        DEC CX
        JNZ UP
        MOV LAR, AX           ; store largest value
        MOV AH, 4CH
        INT 21H
CODE ENDS
END START

```

Smallest Number in a Series

```

DATA SEGMENT
    X    DW 0060H,0020H,0030H,0040H,0050H
    MES DB 10,13,'SMALLEST NUMBER AMONG THE SERIES IS $'
DATA ENDS

CODE SEGMENT
    ASSUME CS:CODE, DS:DATA

START:  MOV AX, DATA
        MOV DS, AX
        MOV CX, 05H
        LEA SI, X
        MOV AX,[SI]           ; AX = running minimum, starts at X[0]
        DEC CX
UP:     CMP AX,[SI+2]          ; compare running min with next element
        JB CONTINUE           ; if AX already smaller, keep it
        MOV AX,[SI+2]         ; else update running min
CONTINUE: ADD SI, 2
        DEC CX
        JNZ UP
        AAM                   ; convert smallest binary value to unpacked BCD

```



```

        ADD AX, 3030H          ; convert unpacked BCD to ASCII
        MOV BX, AX             ; keep ASCII digits in BX
        MOV AH, 09H           ; DOS: display message
        LEA DX, MES
        INT 21H
        MOV DL, BH             ; display tens digit
        MOV AH, 02H
        INT 21H
        MOV DL, BL             ; display units digit
        INT 21H
        MOV AH, 4CH
        INT 21H
CODE ENDS
END START

```

; OUTPUT: SMALLEST NUMBER AMONG THE SERIES IS 20

Week #13 — Lab #13

Sort Numbers — Ascending / Descending (Bubble Sort)

```

DATA SEGMENT
    X    DW 42H, 34H, 26H, 17H, 09H
    LEN  EQU 05
    ASCD DB 10 DUP(0)
DATA ENDS

CODE SEGMENT
    ASSUME CS:CODE, DS:DATA

START:  MOV AX, DATA
        MOV DS, AX
        MOV BX, LEN-1          ; counter1 = passes remaining
        MOV CX, BX             ; counter2 = copy for outer loop
UP1:    MOV BX, CX              ; reload counter1 for this pass
        LEA SI, X
UP:      MOV AX, [SI]            ; AX = element[i]
        MOV DX, [SI+2]          ; DX = element[i+1]
        CMP AX, DX
        JB DOWN                 ; ASCENDING: if AX < DX already in order, skip swap
        ;JA DOWN                ; (use this line instead of JB for DESCENDING order)
        MOV [SI], DX            ; otherwise swap: smaller value first
        MOV [SI+2], AX
DOWN:    INC SI
        INC SI
        DEC BX
        JNZ UP                  ; continue this pass
        DEC CX
        JNZ UP1                 ; do another pass
        MOV AH, 4CH
        INT 21H
CODE ENDS
END START

; OUTPUT (ascending): 09 17 26 34 42

```

String Transfer

```

DATA SEGMENT
    STR1 DB 'HOW ARE YOU'
    LEN  EQU $-STR1
    STR2 DB 20 DUP(0)
DATA ENDS

```

```

CODE SEGMENT
    ASSUME CS:CODE, DS:DATA, ES:DATA

START:  MOV AX, DATA
        MOV DS, AX          ; DS:SI -> source string
        MOV ES, AX          ; ES:DI -> destination string
        LEA SI, STR1
        LEA DI, STR2
        MOV CX, LEN         ; number of characters to move
        CLD                 ; clear direction flag: auto-increment SI/DI
        REP MOVSB           ; copy CX bytes from STR1 to STR2
        MOV AH, 4CH
        INT 21H

CODE ENDS
END START

```

Search for a Character in a String

```

DATA SEGMENT
    MSG DB 'HELLO'
    CNT EQU $-MSG
    SRC EQU 'E'
    MSG1 DB 10,13,'CHARACTER FOUND$'
    MSG2 DB 10,13,'CHARACTER NOT FOUND$'
DATA ENDS

CODE SEGMENT
    ASSUME CS:CODE, DS:DATA, ES:DATA

START:  MOV AX, DATA
        MOV DS, AX
        MOV ES, AX
        LEA DI, MSG         ; ES:DI -> string to search (SCASB uses ES:DI)
        MOV AL, SRC         ; character to search for
        MOV CL, CNT         ; number of characters
        MOV CH, 00H
        CLD                 ; auto-increment DI
UP:     SCASB                ; compare AL with [ES:DI], DI++
        JZ DOWN             ; match found
        LOOP UP              ; else decrement CX and continue
        LEA DX, MSG2
        MOV AH, 09H
        INT 21H
        JMP EXIT
DOWN:   LEA DX, MSG1
        MOV AH, 09H
        INT 21H
EXIT:   MOV AH, 4CH
        INT 21H

CODE ENDS
END START

; OUTPUT: CHARACTER FOUND

```

Week #14 — Lab #14

DOS Keyboard Input and Console Display (INT 21H)

```
DATA SEGMENT
    INKEY DB ?
    BUF   DB 20 DUP(0)
    MES   DB 10,13,'DAYANANDA SAGAR COLLEGE OF ENGINEERING $'
DATA ENDS

CODE SEGMENT
    ASSUME CS:CODE, DS:DATA

START:  MOV AX, DATA
        MOV DS, AX
        MOV AH, 01H           ; DOS function 01H: read character with echo
        INT 21H               ; AL = ASCII code of key pressed
        MOV INKEY, AL

        MOV BUF, 10           ; BUF[0] = max characters to read
        MOV AH, 0AH           ; DOS function 0AH: buffered string input
        LEA DX, BUF
        INT 21H

        MOV AH, 06H           ; DOS function 06H: display single character
        MOV DL, 'A'
        INT 21H

        MOV AH, 09H           ; DOS function 09H: display '$'-terminated string
        LEA DX, MES
        INT 21H

        MOV AH, 4CH
        INT 21H
CODE ENDS
END START
```

Week #15 — Lab #15

Interfacing — Seven-Segment Display (8255 PPI)

```
DATA SEGMENT
    PORTA EQU 120H
    PORTB EQU 121H
    PORTC EQU 122H
    CWRD  EQU 123H
    TABLE DB 8CH, 0C7H, 86H, 89H ; seven-segment codes for digits to display
DATA ENDS

CODE SEGMENT
    ASSUME CS:CODE, DS:DATA

START:  MOV AX, DATA
        MOV DS, AX
        MOV AL, 80H           ; 8255 control word: ports B & C as output
        MOV DX, CWRD
        OUT DX, AL

        MOV BH, 04            ; number of digits to display
```

```

        LEA SI, TABLE          ; SI -> look-up table of segment codes

NEXTDIGIT:
        MOV CL, 08              ; 8 bits (segments) per digit
        MOV AL,[SI]
NEXTBIT:
        ROL AL, 01              ; rotate next segment bit into carry/LSB
        MOV CH, AL              ; save AL
        MOV DX, PORTB
        OUT DX, AL              ; send one segment bit out on port B
        MOV AL, 01
        MOV DX, PORTC
        OUT DX, AL              ; pulse clock line high (PC0)
        DEC AL
        MOV DX, PORTC
        OUT DX, AL              ; clock line low
        MOV AL, CH              ; restore AL
        DEC CL                  ; sent one bit; 7 to go
        JNZ NEXTBIT
        DEC BH                  ; one digit done
        INC SI                  ; next digit's code
        JNZ NEXTDIGIT

        MOV AH, 4CH
        INT 21H

CODE ENDS
END START

```

Interfacing — Stepper Motor (8255 PPI)

```

DATA SEGMENT
        PORTA EQU 120H
        PORTB EQU 121H
        PORTC EQU 122H
        CWRD EQU 123H
DATA ENDS

CODE SEGMENT
        ASSUME CS:CODE, DS:DATA

START:  MOV AX, DATA
        MOV DS, AX
        MOV AL, 80H          ; 8255 control word: port A as output
        MOV DX, CWRD
        OUT DX, AL
        MOV DX, PORTA
        MOV AL, 88H          ; initial excitation pattern

UP:     OUT DX, AL            ; output current pattern to stepper motor
        CALL DELAY           ; wait between steps
        ROL AL, 01H          ; rotate to next excitation pattern (11,22,44,88 sequence)
        JMP UP               ; repeat forever

DELAY:  MOV CX, 0FFFFH       ; outer delay count (adjust for different speeds)
UP2:    MOV BX, 0FFH         ; inner delay count
UP1:    DEC BX
        JNZ UP1
        DEC CX
        JNZ UP2
        RET
        ; MOV AH,4CH / INT 21H -- unreachable: the excitation loop above runs indefinitely

CODE ENDS
END START

```